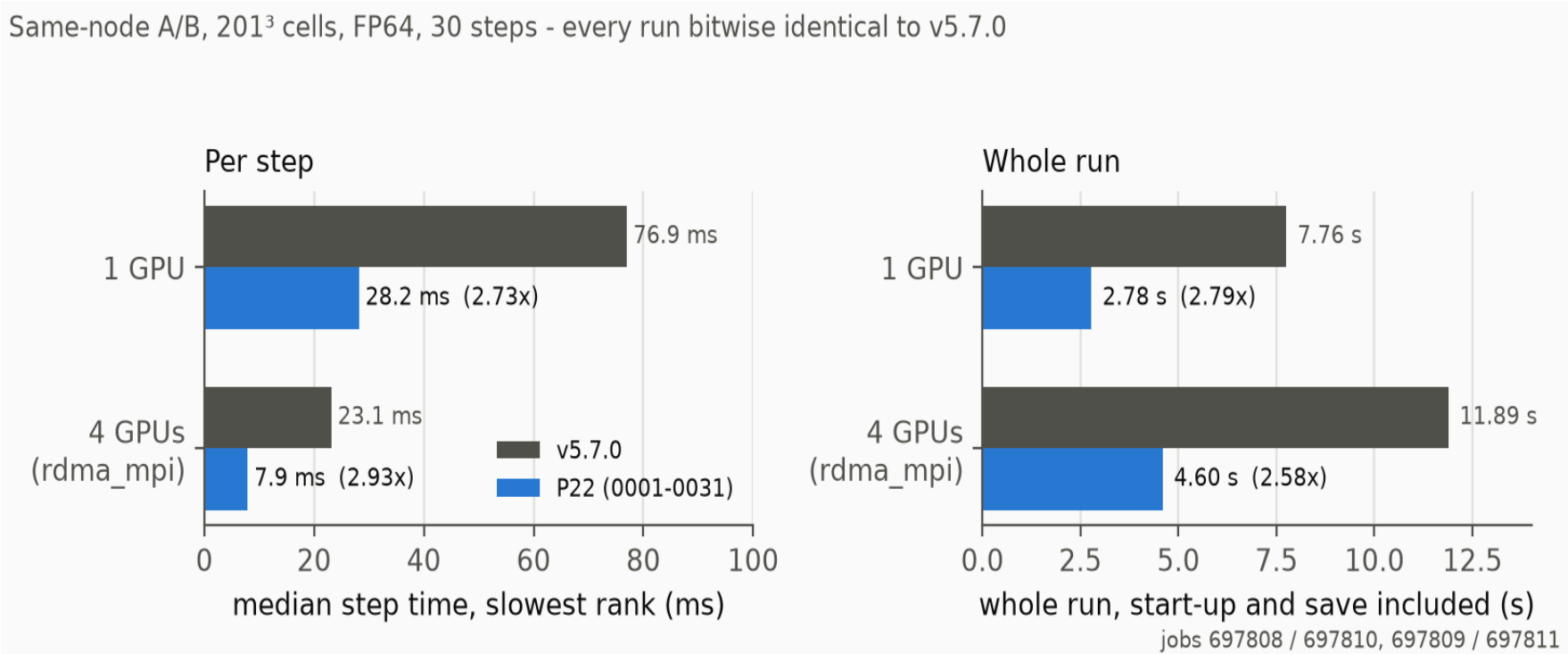
Making MFC faster on H200 without changing its answer

CS5434 High Performance Computing, Coding-Adv-HW-1

MFC v5.7.0, SC26 Student Cluster Competition application, nano4 (8× H200)

Result: 2.7× faster per step, bitwise identical



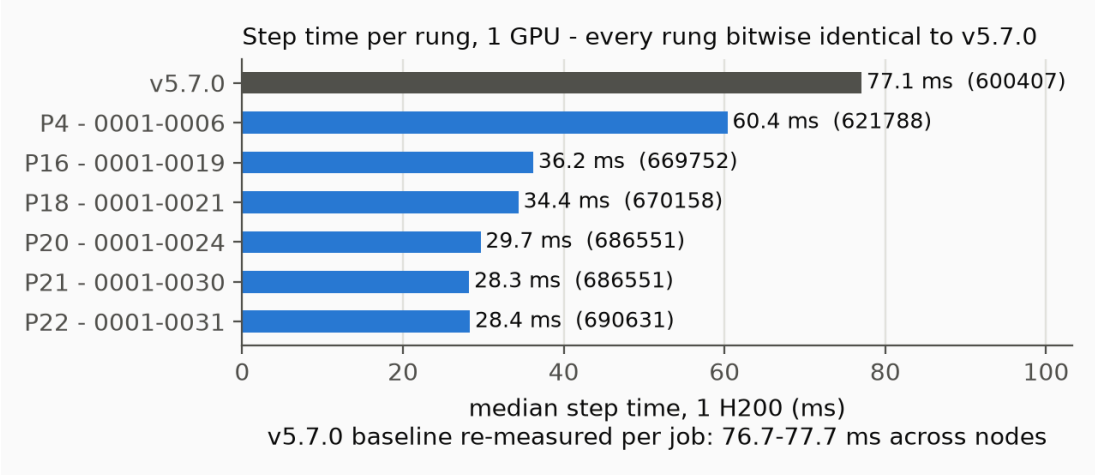
1 GPU: step 76.9 → 28.2 ms (2.73×), whole run 7.76 → 2.78 s (2.79×) · 4 GPUs: 23.1 → 7.9 ms (2.93×), 11.89 → 4.60 s (2.58×) Grind time, MFC's own score figure (ns per grid point per equation per RHS): 0.397 → 0.145

What we changed: 31 patches, no physics touched

Group	Patches	What it does
Stop copying	0007–0009, 0011, 0023, 0024	read an array where it is instead of copying it first
Merge passes	0010, 0016, 0017, 0022	one pass over the grid instead of four
Prefetch	0020, 0021	load the next values while still working on these
Remove host waits	0013, 0019, 0025–0031	the CPU stops blocking on the GPU
Build and flags	0002–0005, 0012, 0014, 0018	register caps, one translation unit

Same equations, same grid, same step count, same FP64. One patch per change, in `patches/opt/`.

Why it is faster



- Profile first: WENO5 34 % and HLLC 25 % of a stage; copy-like kernels ≈25 % at a third of memory bandwidth
- Register caps and one translation unit fixed occupancy and inlining
- Still open: the copy kernels, and the halo exchange does not overlap with compute

Correctness and limits

- Every run: final state bitwise identical to v5.7.0 (65 M numbers)
- Full MFC suite on the final build: 713 tests, 690 pass, 23 fail — exactly the 23 that fail on unmodified v5.7.0, same errors, 0 new
- 7 more patches measured but not suite-validated (`patches/pending/`); the headline does not use them
- The 10-31 report's 8-GPU number compares v5.7.0 at its defaults against our `rdma_mpi` , so it is asymmetric
- Next: overlap halo with compute, fuse the copy kernels, send the patches upstream

Report `report.md` , patches `patches/opt/` , raw data `results/`